

Esercizio A1 – Complessità – Punti ≥ 8

Per risolvere il problema, si può utilizzare il master theorem, notando che $a = 1$, $b = 1/\delta$, $\beta = \delta + 1$. Ignorando l'operatore $\lfloor \rfloor$, si ha che $\alpha = \log_b a = 0$. Siamo quindi nel caso (3) ($\alpha < \beta$) e quindi la complessità è pari a $\Theta(n^\beta) = \Theta(n^{\delta+1})$.

Esercizio A2 – Unlimited – Punti ≥ 8

L'algoritmo utilizza una visita in ampiezza per esplorare tutte le celle raggiungibili a partire da *start*, considerando come archi del grafo le mosse *forward* e *backward*.

Durante la visita, ogni elemento della coda è una coppia $(x, dist)$, dove x rappresenta una cella raggiunta e $dist$ il numero di mosse effettuate per raggiungerla.

La ricerca prosegue fino a quando la cella 1 viene raggiunta, nel qual caso viene restituito il valore d corrispondente. Poiché la visita in ampiezza esplora tutti i percorsi più brevi prima di quelli più lunghi, il valore restituito rappresenta il numero minimo di mosse necessarie.

L'uso dell'insieme *visited* garantisce che ogni cella venga visitata al più una volta, evitando cicli e ridondanze.

Se la cella 1 non è raggiungibile, la coda si svuota senza che venga mai soddisfatta la condizione $x = 1$, e l'algoritmo restituisce -1 come richiesto.

Ogni cella x viene aggiunta alla coda al più una volta, e per ogni cella si effettuano al massimo due controlli (uno in avanti, uno indietro). La complessità complessiva è quindi $O(n)$.

```

int minSteps(boolean [] A, int n, int start, int f, int b)
{
    QUEUE Q = Queue()
    SET visited = Set()
    Q.enqueue((start, 0))
    visited.insert(start)
    while not Q.isEmpty() do
        (int, int)(x, dist) = Q.dequeue()
        if x == 1 then
            return dist
        if x - b ≥ 1 and A[x - b] and (x - b ∉ visited) then
            Q.enqueue((x - b, dist + 1))
            visited.insert(x - b)
        if x + f ≤ n and A[x + f] and (x + f ∉ visited) then
            Q.enqueue((x + f, dist + 1))
            visited.insert(x + f)
    return -1
}

```

Soluzione basata su una visita in profondità In un primo momento ho valutato errate le soluzioni basate su una visita in profondità, poiché, in un grafo arbitrario, una visita in profondità (*depth-first search*, DFS) può raggiungere la destinazione in un numero di mosse superiore rispetto a una visita in ampiezza (*breadth-first search*, BFS), che garantisce invece il cammino minimo. Tuttavia, nel caso specifico di questo esercizio, il grafo generato ha una struttura particolare, in cui la destinazione è fissata nella cella 1, e le mosse consentite sono orientate (in gran parte) verso sinistra.

Se avessi scelto una destinazione generica *dest*, avrei avuto ragione a escludere la DFS come strategia corretta per trovare il cammino minimo. La decisione di fissare la destinazione in 1 era motivata dalla volontà di non complicare la consegna, ma si è rivelata fonte di ambiguità in fase di correzione, perché la correttezza della DFS dipende dall'ordine con cui vengono esplorate le mosse.

- **DFS con ordine *forward*-prima.** Se nella soluzione la DFS esplora prima la mossa *forward* (cioè verso destra) e solo dopo quella *backward*, la soluzione può essere subottimale. Ad esempio, con $n = 6$, $start = 5$, $f = 1$, $b = 2$, e tutte le celle percorribili, la DFS visita le celle in questo ordine:

$$5 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1.$$

Nota: da 6 non può andare a 7 (fuori limite), da 4 non può tornare a 5 (già visitata), e da 3 non può tornare a 4 (già visitata). Il cammino più breve sarebbe invece:

$$5 \rightarrow 3 \rightarrow 1,$$

che richiede solo due mosse. In questo caso, la DFS ha esplorato un ramo non ottimale a causa dell'ordine errato delle chiamate ricorsive.

- **DFS con ordine *backward*-prima.** Se invece la DFS esplora prima la mossa *backward* (cioè verso sinistra) e solo dopo la mossa *forward*, allora il primo cammino che raggiunge la cella 1 è sempre ottimo, nonostante la presenza di elementi "bloccati". Informalmente, questo avviene perché se in un percorso minimo ci sono mosse forward, queste possono essere posticipate il più tardi possibile, dando precedenza alle mosse backward. Appena riesco a scrivere una dimostrazione decente, la aggiungo qui – non è facile scriverla in maniera scorrevole.

In fase di correzione, mi sono trovato in un dilemma: gli algoritmi basati su visita in profondità backward-forward sono corretti, ma gli studenti che li hanno scritti avrebbero dovuto dare una dimostrazione e non l'hanno fatto. Avrebbero dovuto fare come gli altri, ovvero "buttarsi" su una visita in ampiezza, ricordandosi quanto detto a lezione: le visite in profondità non garantiscono la distanza minima. Hanno quindi preso 90%, per mancanza di dimostrazione. Gli algoritmi basati su visita in profondità forward-backward non sono corretti, e hanno preso 50%.

Soluzione errata: ricorsione senza *visited* Alcune persone hanno semplicemente scritto una soluzione ricorsiva che, quando viene chiamata sulla cella i , visita le celle $i - b$ e $i + f$ ricorsivamente, facendo gli opportuni controlli sui margini e sulle celle visitabili, ma senza utilizzare un insieme *visited*. L'assenza di tale vettore fa sì che la visita possa andare avanti in eterno, perché l'algoritmo può seguire cicli in continuazione. Per esempio, si consideri questa configurazione, con $start = 6$, $b = 1$ e $f = 1$.

1	2	3	4	5	6	7	8	9	10
			F	T	T	T	F		

Si può vedere che la visita continuerà nelle celle 5,6,7 in eterno, restando bloccata all'interno.

Esercizio A3 – Stringhe pareggiate – Punti ≥ 8

Un primo approccio per risolvere il problema consiste nel generare tutte le possibili sottostringhe della stringa di input e , per ciascuna di esse, verificare se è *pareggiata*, ovvero se per ogni lettera presente è presente anche la corrispondente lettera con capitalizzazione opposta.

Per farlo, possiamo definire due funzioni:

- $\text{largestPaired}(S, n)$: genera tutte le sottostringhe e tiene traccia della lunghezza massima fra quelle pareggiate.
- $\text{isPaired}(S, start, end)$: restituisce **true** se la sottostringa $S[start \dots end]$ è pareggiata.

```

int largestPaired(item[] S, int n)
{
    int maxSoFar = 0
    for start = 1 to n do
        for end = start to n do
            if isPaired(S, start, end) and end - start + 1 > maxSoFar then
                maxSoFar = end - start + 1
    return maxSoFar

```

Un'implementazione ingenua della funzione $\text{isPaired}()$ confronta ogni carattere con tutti gli altri:

```

boolean isPaired(item[] S, int start, int end)
{
    for i = start to end do
        boolean found = false
        for j = start to end do
            if S[j] = swapcase(S[i]) then
                found = true
        if not found then
            return false
    return true

```

Con questa implementazione, generiamo $\Theta(n^2)$ sottostringhe, e per ciascuna facciamo $\Theta(n^2)$ confronti, per un costo totale $\Theta(n^4)$.

Ottimizzazione con insiemi È possibile migliorare significativamente la funzione `isPaired()` usando un insieme basato su tabella hash per memorizzare i caratteri della sottostringa. Per ciascun carattere c presente, verifichiamo in tempo costante se è presente anche `swapcase(c)`.

```
boolean isPaired(item[] S, int start, int end)
```

```
    SET letters = Set()
    for i = start to end do
        letters.insert(S[i])
    foreach c in letters do
        if not letters.contains(swapcase(c)) then
            return false
    return true
```

Questa ottimizzazione riduce il costo della verifica da $\Theta(n^2)$ a $\Theta(n)$, portando la complessità complessiva dell'algoritmo da $\Theta(n^4)$ a $\Theta(n^3)$.

Approccio divide-et-impera Per ottenere una soluzione più efficiente, osserviamo che se un carattere c compare nella sottostringa $S[i \dots j]$ ma `swapcase(c)` non compare, allora *nessuna sottostringa che lo contiene può essere pareggiata*. Possiamo quindi dividere il problema in due sottoproblemi escludendo c .

```
int largestPaired(item[] S, int n)
```

```
    return lpRec(S, 1, n)
```

```
int lpRec(item[] S, int i, int j)
```

```
    if i > j then
        return 0
    HASH letters = Hash()
    for k = i to j do
        letters[S[k]] = k
    foreach c in letters do
        if not letters.contains(swapcase(c)) then
            int m = letters[c]
            return max(lpRec(S, i, m - 1), lpRec(S, m + 1, j))
    return j - i + 1
```

Nel caso peggiore (ogni carattere è sbilanciato), l'algoritmo esegue una divisione di un carattere per volta, e il costo è:

$$T(n) = T(n - 1) + n = \Theta(n^2)$$

Nel caso migliore (la stringa è interamente pareggiata), la funzione restituisce subito la sua lunghezza, con costo $\Omega(n)$. Quindi la complessità complessiva è compresa tra $\Omega(n)$ e $\Theta(n^2)$.

Esercizio B1 – Braccia rubate alla ricerca – Punti ≥ 8

Idea generale. Il problema può essere modellato come un problema di *flusso massimo* in un grafo bipartito, in cui colleghiamo ogni persona agli uffici che è disposta a pulire, rispettando il vincolo massimo m sul numero di uffici assegnabili a ciascuna persona.

Costruzione del grafo. Costruiamo un grafo orientato $G = (V, E)$ con i seguenti nodi e archi:

- un nodo sorgente s ;
- un nodo pozzo t ;
- un nodo per ogni persona $\ell \in L$;
- un nodo per ogni ufficio $o \in O$;

Colleghiamo i nodi come segue:

- da s a ogni persona $\ell \in L$, un arco con capacità m ;